

LIBRARY MANAGEMENT SYSTEM

(VITYarthi Python Essentials Project)

- Submitted by -
Atharva Sharma
26BCE11003
B. Tech/CSE core

University-VIT Bhopal University

1. INTRODCUTION

The library management system is a python-bases application designed to manage basic library activities. The system allows the user to add and view books, search for books add and view students, issue and return books and generate a library report

The project is divided into different python modules such as books, students, transactions, validation, storage and reports. This modular structure makes the program easier to understand maintain and modify

The system also provides data storage using a text file so that books students and issued -books information can be saved and loaded when the program is run again

2. PROBLEM STATEMENT

Managing books and student records manually can be time-consuming and may lead to errors. It can also become difficult to keep track of which books are available and which books have been issued

The purpose of this project is to develop a simple Library Management System project that stores book and student information and manages book issuing and returning through a Python program

3. FUNCTIONAL REQUIREMENTS

This is basically what our programme can do

Our programme can-

- Add a book
- Search a book
- Issue a book
- Add a student id
- View a student id
- Return book
- Prevent book from issuing if its already issued
- Save library data
- View al available books which can be issued
- View currently issued books

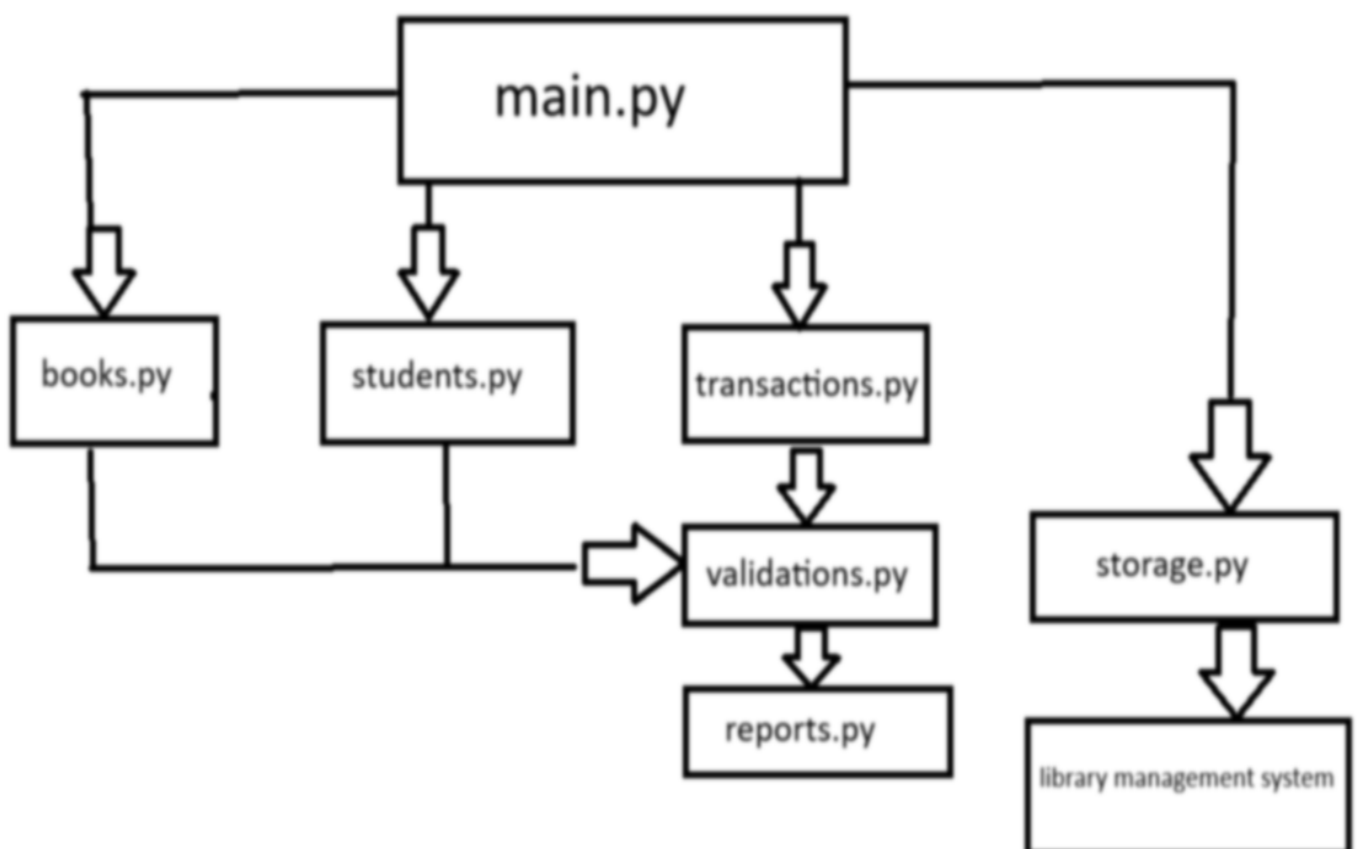
4. NON- FUNCTIONAL REQUIRMENTS

These are conditions of the system

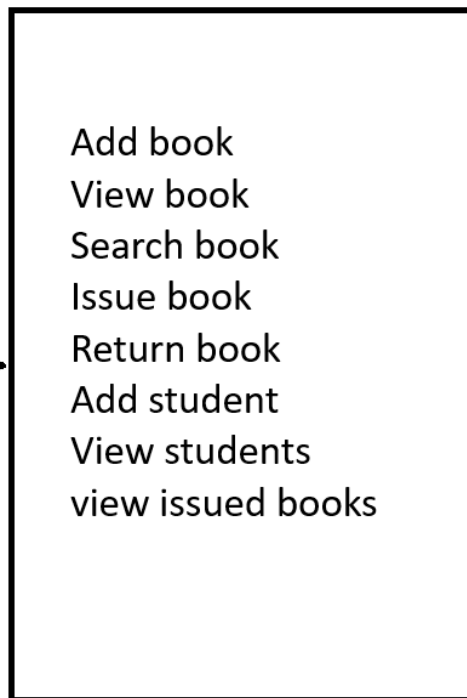
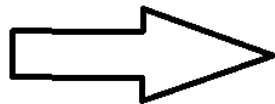
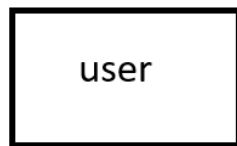
- Usability-The system has a simple menu-bases interface
- Reliability-The system checks whether books and students exist before issuing a book.
- Maintainability-The program is divided into separate python modules
- Dara persistence-data is stored in a text file so it can be loaded later
- Simplicity-the system uses basic python file so it can be loaded later

5. SYSTEM ARCHITECTURE

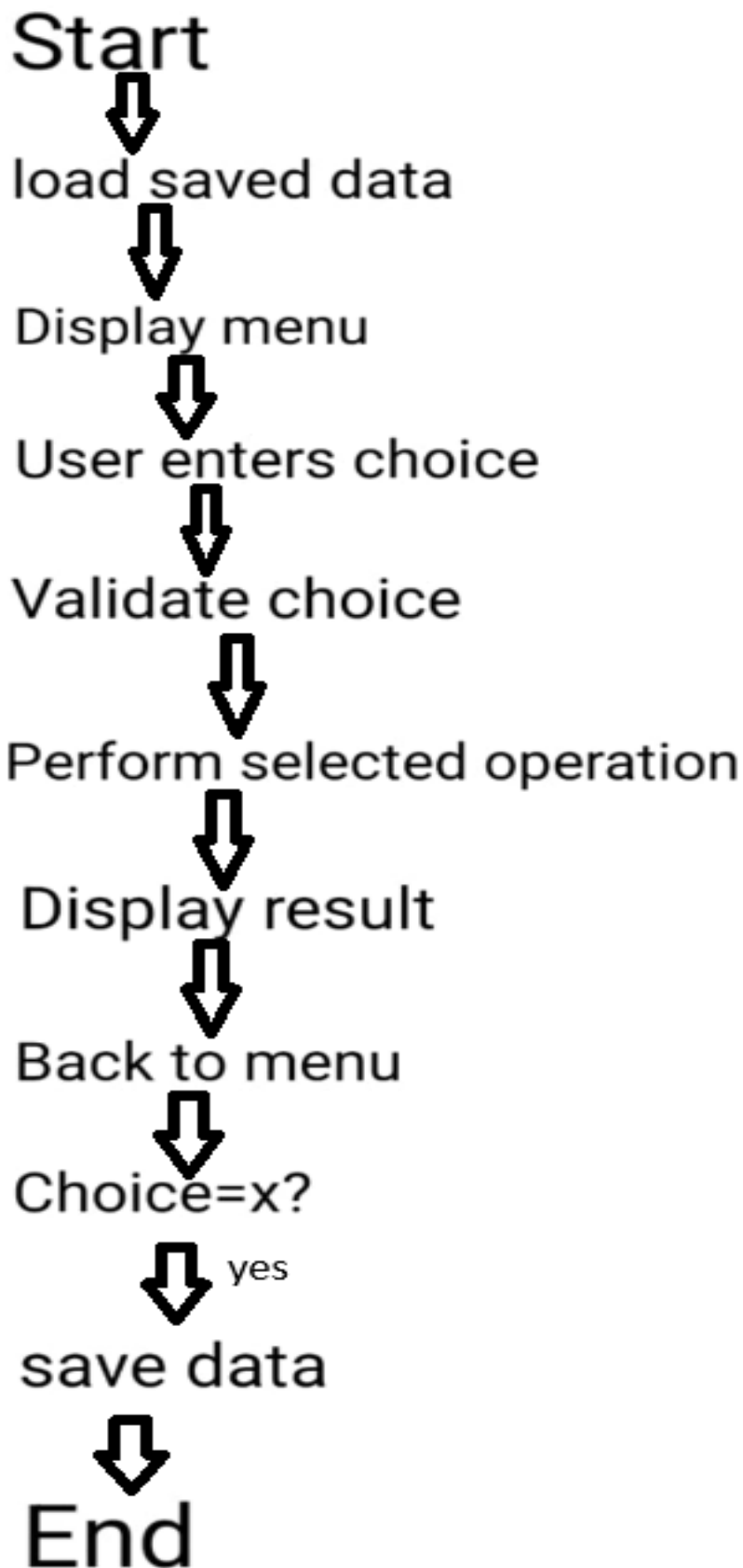
The main.py file act as the main controller and imports functions from different modules. The books.py module manages books students.py manages students records transactions.py handles issuing and returning books validation.py validates menu choices, reports.py generates reports and storage.py handles saving and loading data



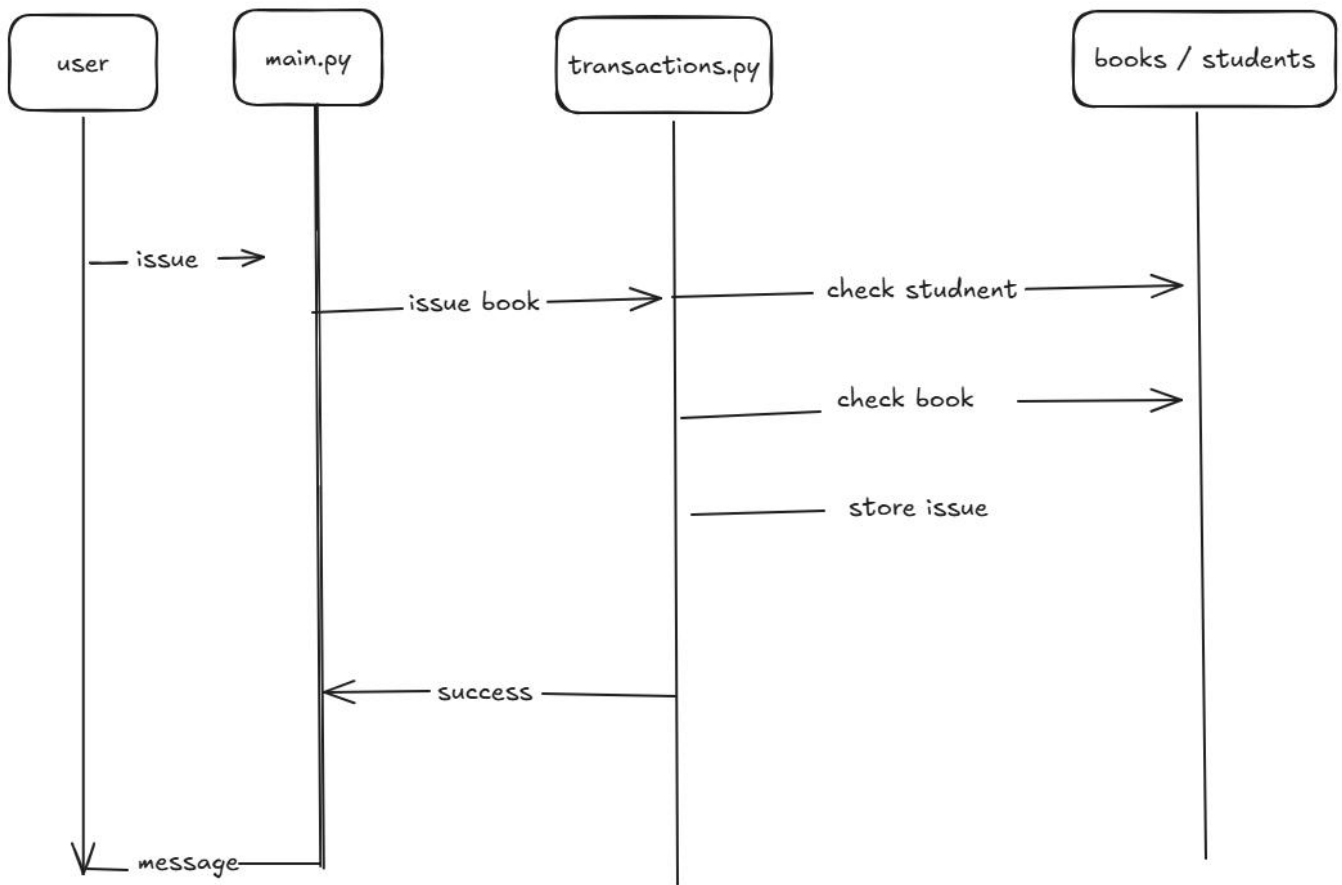
6. USE CASE DIAGRAM



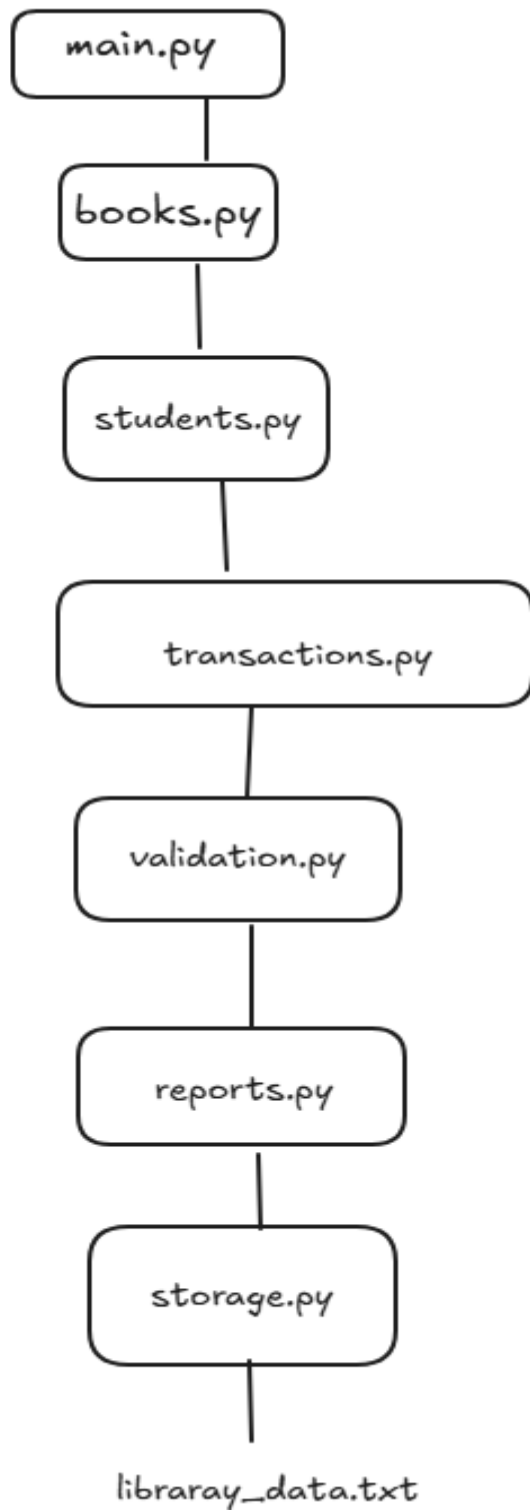
7. WORK FLOW DIAGRAM



8. SEQUENCE DIAGRAM



9. COMPONENT DIAGRAM



10. DESIGN DECISIONS AND RATIONALE

- The program was divided into multiple modules to keep different responsibilities separate
- Lists were used to store books, students and issued-book records
- Book ids and student ids were used to identify records
- A text file was used for data persistence
- Functions were used to perform individual operations
- Menu validation was separated into its own module
- The system checks whether a book or student exists before issuing a book

11. IMPLEMENTATION DETAILS

- Main.py consists (# is used as py words aren't in dictionary)
Controls the menu and calls functions from other modules
- Books.py contains
#add_book()
#view_books()
#search_book()
- Student.py contains
#add_student()
#view_student()
#find_student()
- Transactions.py contains
#Issue_book(), # return_book(), # view_issued_books()
- Validation.py
Checks whether the entered menu choice is valid
- Reports.py
Calculates totals such as #total_books, total students
- Storage.py uses the following to retrieve information
save_data
Load_data

12. Screenshots/results

- Menu

```
==LIBRARY MANAGEMENT SYSTEM==  
1. Add Book  
2. View Books  
3. Search Book  
4. Issue Book  
5. Return Book  
6. Add Student  
7. View Students  
8. View Issued Books  
9. Library Report  
10. Exit  
Enter your choice: █
```

- Adding a book

```
Enter your choice: 1  
Enter book ID: 3340  
Enter book name: jjk manga  
Enter author name: kashimoto  
Book added successfully!
```

- viewing books

```
===== BOOKS =====  
ID: 3340  
Name: jjk manga  
Author: kashimoto
```

- adding a student

```
Enter your choice: 6  
Enter student ID: 26bce11003  
Enter student name: Atharva sharma  
Student added successfully!
```

- Issue a book

```
Enter your choice: 4
Enter student ID: 26bce11003
Enter book ID: 3340
Book issued successfully!
```

- Returning a book

```
Enter your choice: 5
Enter student ID: 26bce11003
Enter book ID: 3340
Book returned successfully!
```

- Library report

```
===== LIBRARY REPORT =====
Total books: 1
Total students: 1
Books currently issued: 0
```

- Viewing students

```
===== STUDENTS =====
ID: 26bce11003
Name: Atharva sharma
-----
```

13. Testing approach

Test	input	expected result
Add a book	valid book details	Book is added
Search book	existing book id	book details displayed
Search book	invalid book id	“book not found”
Add student	valid student details	student added
Issue books	valid student and book id	book issued
Issue book	invalid student ID	student not found
Issue book	same book ID	book not found
Issued book	Valid issue record	book is already issued
Issued book already	same book id	book is already issued
Return book	valid issue record	book returned

Invalid menu	11	choice rejected
Exit	10	data saved and program closes

14. Challenges faced

- During the development of the project several challenges were encountered. One challenge was organizing the program into multiple python modules and making them work together correctly.
- Another challenge was managing book and student ids while issuing and returning books. The system needed to check whether the entered student and book existed before creating an issue record
- Implementing data persistence was also a challenge because the program needed to save information to file and load it again while the program restarted
- One of the biggest challenges was to make this project

16. LEARNING AND KEY TAKEAWAYS

- Learning how to divide a python project into modules
- Learned how lists can store multiple records
- Learned how different modules can work together
- Learned how to test different cases in a program
- Improved understanding of project organization and documentation
- Learned how to read and write files
- Learned how to use loops and conditional statements

17. FUTURE ENHANCEMENTS

- Add a graphical user interface
- Add book availability status
- Add a good calculation system
- Add more data storage
- Add options to update records
- Add options to delete records
- Add due dates and late return dates
- Add a fine system for the students who returned their book late

18. REFERENCES

- Vs code documentation
- Class notes
- Vityarthi lectures